MODULE 17 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

The Systems Life Cycle

Plan, build, deploy, operate, maintain and retire a Claude feature: monitoring, incidents, model retirements, API version migrations, and changing what others depend on.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 17 of 36

admin@sthithapragna.com

Owning a car: buy it, drive it, service it, pass the MOT, part-exchange it. **Nobody plans only the purchase.**

The life cycle of a Claude feature

01

Plan

Requirements and success criteria. Module 9.

02

Build

Design, tests, evals. Modules 10 and 15.

03

Deploy

Configuration, canary, rollback. Module 16.

04

Operate

Monitor quality, cost and errors; handle incidents.

05

Maintain, retire

Upgrades, migrations, and a planned end.

THE TAKEAWAY

Each phase hands the next one something it needs: criteria, tests, flags, metrics, a migration plan.

What to watch in production

- **Quality:** sampled outputs graded against the success criteria; user corrections and complaints.
- **Signals of trouble:** validation failures, refusal and max_tokens stop reasons, fallbacks taken.
- **Service health:** error rates, 429s, latency and time to first token.
- **Cost per unit:** per conversation, per ticket, per document, from the usage fields.
- **Slice by prompt_version and model,** so a change is visible the day it ships.

> ON THE EXAM
>
> Uptime says the service answered. Only quality monitoring says the answers were good.

When something goes wrong

- **A kill switch:** a flag that turns the feature off, or back to the last good version, without a deploy.
- **A fallback path:** a person, a simpler rule, or a clear message instead of a broken answer.
- **A runbook** the on-call engineer can follow at 3am.
- **Request IDs and prompt versions** in the logs, to trace one bad answer to its cause.
- **Afterwards:** the failing case joins the regression set.

> THE TIEBREAKER
>
> Design the off switch before you need it. Incidents are a designed path, not an improvisation.

Model life cycle states

STATE	WHAT IT MEANS
Active	Fully supported and recommended
Legacy	No longer updated; may be deprecated later
Deprecated	Still works; replacement named; retirement date set
Retired	No longer available. Requests fail

ON THE EXAM

At least 60 days' notice before a publicly released model retires. Sonnet 4.5 retires on 30 November 2026.

Migrating off a retiring model

01
Audit
Console usage export, by API key and model.

02
Test
Replacement on the regression set, well before the date.

03
Fix
Prompt and code changes the new model needs.

04
Migrate
Validated batches, behind flags.

05
Confirm
Usage on the old ID reaches zero.

THE TAKEAWAY

Retirement is a planned event with a date. Put it on the roadmap the day the notice arrives.

API version 2026-10-21

- **temperature, top_p and top_k are removed:** sending any of them returns 400, "even at the parameter's default value".
- **A document block in an assistant message** returns 400: send documents in user messages.
- **Citations on user text blocks** return 400.
- **Opt in:** anthropic-version: 2026-10-21, or a new SDK major version, such as Python anthropic 2.0.0.
- **On Bedrock,** the new version is available only for Claude 4.7 and later.

> CURRENCY CHECK
>
> Anthropic: "Release the upgrade together with your fixes." Version changes are migrations, not upgrades you click.

Copperlet: the parameter in forty places

BEFORE

A relic, everywhere

- temperature passed at about 40 call sites
- Left over from older models
- Will return 400 under the new API version
- Nobody sure which calls rely on it

AFTER

Migrated in batches

- Inventory from code search and request logs
- Removed service by service, evals run each time
- A client-module test rejects sampling parameters
- SDK upgrade released with the fixes

SETTLED

Quirk 9 settled. Consistency now comes from prompts, schemas and validation, as module 7 taught.

Changing what others depend on

- **Know your consumers:** services, reports, agents and teams that read your output or call your API.
- **Give notice,** with a deprecation window, a named replacement and a date, as Anthropic does.
- **Run old and new side by side** during the window; watch usage of the old path fall.
- **Retire deliberately:** switch off, then delete stored data by the retention policy.
- **Record the decision,** so the next team knows why it changed.

> THE TAKEAWAY
>
> Treat your own feature's consumers the way you want Anthropic to treat you.

A support assistant has run for six months. The team monitors uptime and error rates, which look healthy. After the company launched a new product line, complaints say answers have become vague and sometimes wrong.

Nothing in the team's dashboards changed.

What should the team add?

A More frequent uptime checks from several regions

B Quality monitoring on sampled production answers, failures and corrections

C An immediate switch to the newest model available

D A rerun of the original pre-launch eval set, unchanged

◆ PAUSE AND DECIDE

What should the team add?

A More frequent uptime checks from several regions

Uptime is already fine. Quality is not.

B Quality monitoring on sampled production answers, failures and corrections

Sees what users see, as inputs change.

C An immediate switch to the newest model available

A change before any evidence.

D A rerun of the original pre-launch eval set, unchanged

Has none of the new product line in it.

THE SIGNAL

"Dashboards fine, users unhappy": you are monitoring service health, not answer quality.

> A company receives notice that a model used by several of its services will retire in about 60 days. Nobody is sure which services, API keys or teams still call it.
>
> A replacement model is named in the notice.

What is the best plan?

A Wait until the final week, when the replacement will be better understood

B Switch every service to the replacement today, without testing

C Audit usage, test the replacement, then migrate in validated batches

D Keep the old model ID, since retired models redirect to their replacement

◆ PAUSE AND DECIDE

What is the best plan?

A Wait until the final week, when the replacement will be better understood

No time left to test or fix.

B Switch every service to the replacement today, without testing

Untested change, all at once.

C Audit usage, test the replacement, then migrate in validated batches

Anthropic's documented steps.

D Keep the old model ID, since retired models redirect to their replacement

They do not. Requests fail.

THE SIGNAL

"Nobody is sure who calls it": audit first. Then test, then migrate in batches.

> A team is moving to API version 2026-10-21. Its code sends temperature at many call sites, and its few-shot examples place document blocks inside assistant messages.
>
> The team wants a smooth migration.

Which two steps are correct? Choose two.

- A Remove the sampling parameters, relying on prompts and validation
- B Keep temperature, set to its default value, which is still accepted
- C Stay on the old SDK and API version permanently
- D Move document blocks into user messages, then test and release together
- E Upgrade the SDK first, and fix errors as they appear in production

◆ PAUSE AND DECIDE

Which two steps are correct? Choose two.

A Remove the sampling parameters, relying on prompts and validation

The parameters are gone from the new version.

B Keep temperature, set to its default value, which is still accepted

400 "even at the parameter's default value".

C Stay on the old SDK and API version permanently

Defers the work and falls behind.

D Move document blocks into user messages, then test and release together

Fixes the second break, the documented way.

E Upgrade the SDK first, and fix errors as they appear in production

Customers find the 400s.

CURRENCY CHECK

Two breaking changes, two fixes, released together with the upgrade.

> A new prompt version starts producing harmful drafts at 2am. The on-call engineer cannot turn the feature off without a code deploy, which needs a second approver who is asleep.
>
> The team wants the next incident to be contained in minutes.

What should the team build?

A. A larger on-call rota, so that an approver is always awake

B. A freeze on all prompt releases until the cause is fully understood

C. An instruction for Claude to detect and correct its own harmful drafts

D. A kill-switch flag and a known-good fallback version, with a runbook for on-call

◆ PAUSE AND DECIDE

What should the team build?

A. A larger on-call rota, so that an approver is always awake

More people, same slow path.

B. A freeze on all prompt releases until the cause is fully understood

Prevents change, not damage.

C. An instruction for Claude to detect and correct its own harmful drafts

Guidance, at the moment it is failing.

D. A kill-switch flag and a known-good fallback version, with a runbook for on-call

Contained in minutes, no deploy.

THE SIGNAL

"Cannot turn it off without a deploy" is the signal: build the off switch in advance.

A company is retiring an old document summariser in favour of a new one. Two other services still call the old endpoint, and its database holds summaries containing personal data.

The retention policy says such data must be deleted when no longer needed.

What is the best way to retire it?

A Give notice, migrate both consumers, switch off, then delete the data

B Switch it off today, since the new summariser already exists

C Leave it running indefinitely, in case anyone still needs it

D Delete the code, but keep the stored summaries in case they are useful later

◆ PAUSE AND DECIDE

What is the best way to retire it?

A Give notice, migrate both consumers, switch off, then delete the data

Consumers safe, data handled, a clean end.

B Switch it off today, since the new summariser already exists

Breaks two services without warning.

C Leave it running indefinitely, in case anyone still needs it

Cost, risk and personal data, forever.

D Delete the code, but keep the stored summaries in case they are useful later

Breaks the retention policy.

THE SIGNAL

Retirement covers consumers and data, not just the code.

> After a prompt release, an assistant's average cost per conversation doubled, because the new prompt asked for longer answers. Nobody noticed until the monthly invoice arrived.
>
> The team wants to catch this within a day next time.

What should the team put in place?

A. A quarterly review of the invoice with finance

B. A permanent move to the cheapest model, whatever the quality

C. Daily cost per conversation from the usage fields, by prompt version, with a budget alert

D. A hard cap of one hundred output tokens on every response

◆ PAUSE AND DECIDE

What should the team put in place?

A A quarterly review of the invoice with finance

Slower than the invoice that already failed.

B A permanent move to the cheapest model, whatever the quality

Trades quality for a monitoring gap.

C Daily cost per conversation from the usage fields, by prompt version, with a budget alert

Seen within a day, tied to the cause.

D A hard cap of one hundred output tokens on every response

Truncates good answers everywhere.

THE SIGNAL

Cost per unit, sliced by version, alerting daily. Module 19 builds the cost model.

Module 17 in five lines

- **Shipping starts the work:** operate, maintain and retire are planned phases.
- **Monitor quality and cost,** not just uptime, sliced by prompt version and model.
- **Design incidents:** kill switch, fallback, runbook, regression case.
- **Retirements and API versions** are migrations: audit, test, fix, batches.
- **Retire your own features** with notice, migrated consumers and deleted data.

COMING UP

Next, Part 4: choosing the model, and surviving new releases.